Advanced Quantitative Research Methods

Unsupervised Learning

Advanced module A8 — optional self-study

Prof. Dr. Christoph Weisser

Slides, lab notebooks and data:
github.com/ChrisW09/Advanced-Quantitative-Research-Methods

The advanced modules at a glance

Module	Topic
A1	Randomised Controlled Trials — potential outcomes, randomisation, power
A2	Explainable AI with Shapley Values — axioms, exact and Monte-Carlo Shapley
A3	Conformal Prediction — split conformal, CQR, prediction sets
A4	GLMs and Splines — exponential family, overdispersion, P-splines
A5	Support Vector Machines — margins, the cost C, kernels
A6	Survival Analysis — censoring, Kaplan–Meier, Cox
A7	Multiple Testing — FWER, Bonferroni and Holm, FDR
A8	**Unsupervised Learning — PCA, K-means, hierarchical clustering**
A9	Deep Learning — MLPs, convolutions, training, regularisation

Twelve optional, self-study modules sit outside the taught chapter plan (the table shows A1–A9; A10–A12 are newer additions — see the folder README); today's module is highlighted. Each is a full deck in the course house style with a companion lab notebook.

This module assumes Chapter 2 (what a test error buys you) and Chapter 5 (resampling and stability). It was ISLP Chapter 12 in earlier editions of this course.

Contents

1. No Response
2. PCA
3. K-means
4. Hierarchical
5. Practice
6. Python Lab
7. Summary

Optional and advanced material is collected in the **appendix** at the end of the deck.

Notation in this chapter

Symbol	Meaning
n, p	number of observations; number of features (no response Y anywhere)
x_{ij}	value of feature j for observation i, always *centred* (mean 0)
ϕ_{jm}	loading of feature j on the m-th principal component
$\phi_m = (\phi_{1m}, \ldots, \phi_{pm})^\top$	the m-th loading vector, normalised to $\|\phi_m\| = 1$
z_{im}	score of observation i on component m; $z_{im} = \sum_j \phi_{jm} x_{ij}$
λ_m	variance of the m-th score vector (the m-th eigenvalue)
PVE_m	proportion of variance explained by component m
M	number of components retained
K	number of clusters
$C_1, \ldots, C_K$	the clusters: a partition of $\{1, \ldots, n\}$; $\|C_k\|$ is the size of C_k
$\bar{x}_{kj}$	mean of feature j inside cluster C_k (the j-th centroid coordinate)
$W(C_k)$	within-cluster variation of C_k; $W = \sum_k W(C_k)$
$d(i, i')$	dissimilarity between observations i and i'
$d(A, B)$	linkage: dissimilarity between two *groups* A and B
$r_{ii'}$	correlation between the feature profiles of observations i and i'
s_i	silhouette width of observation i

Sources and references

Primary textbook

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning, with Applications in Python.* Springer.

Data used in this chapter

`USArrests` — four crime and urbanisation measures for the 50 US states; the textbook's PCA example. `Ch12Ex13` — 1000 genes measured on 40 tissue samples (20 healthy, 20 diseased), the textbook's high-dimensional clustering exercise.

Learning objectives

By the end of this chapter you will be able to:

- **Explain** why unsupervised learning has no test error, and what that costs you.
- **Apply** principal component analysis, and read loadings, scores, a biplot and a scree plot.
- **Decide** whether to standardise the features — and show what happens if you do not.
- **Run** K-means, and defend the result against its local-optimum problem.
- **Build** and cut a dendrogram, choosing a linkage and a dissimilarity measure on purpose.
- **Avoid** reporting clusters found in noise as if they were findings.

Roadmap of this chapter

Structure without a supervisor

1. The hard part: no response, so *no test error to validate against*.
2. Principal component analysis: fewer variables, honestly reported.
3. K-means: partition into K groups, and the local-optimum trap.
4. Hierarchical clustering: dendrograms, linkage, dissimilarity.
5. Practice: scaling, outliers, clusters in pure noise, honest reporting.

Outline

1 No Response
2 PCA
3 K-means
4 Hierarchical
5 Practice
6 Python Lab
7 Summary

From supervised to unsupervised: exactly what changes

	Supervised (Ch. 2–11)	**Unsupervised (this chapter)**
Data	(x_i, y_i), $i = 1, \ldots, n$	x_i only
Goal	predict Y from X	describe the structure of X
Loss	MSE, error rate, deviance	no loss involving a truth
Validation	test set, cross-validation	**nothing comparable**
"Right answer"	exists, is withheld, is checkable	does not exist in the data
Output	a prediction rule	a picture, an index, a partition

Takeaway

Only one row matters today: the **validation** row. Everything difficult about this chapter follows from it.

The defining difficulty: there is no test error

Why cross-validation cannot be rescued here

Cross-validation works because a held-out y_i is a *fact* the model did not see. With no y, there is nothing to hold out that can contradict the answer:

$$\text{no } y \implies \text{no } \widehat{\text{Err}} \implies \text{no way to say the answer was } \textit{wrong}.$$

What the methods will still happily give you

K-means returns K clusters for any K and any data. PCA returns p components for any data. Hierarchical clustering returns a full tree for any data. *None of them can fail*, and none of them reports a failure.

Takeaway

Every result in this chapter is a **hypothesis about structure**. The verification happens outside the data set: replication, a designed experiment, or a downstream supervised task whose accuracy you *can* measure.

What we give up — and what we can still borrow (Ch. 2 bridge)

Lost from the Chapter 2 discipline

- **The test-set contract.** No number tells us we have overfitted.
- **Model selection.** K, M, the linkage and the distance cannot be chosen by minimising a validated error.
- **Bias–variance language.** There is no target to be biased about.

Kept from it

- **Stability** still means something: re-fit on a bootstrap sample or a random half and see how much the answer moves.
- **A downstream supervised task** converts the question into a measurable one — do PCA scores predict churn better than the raw features?
- **Honest reporting** of every choice made, because none of them was forced by the data.

So why do it at all? Two families of method

Dimension reduction — PCA

Replace p correlated features by $M \ll p$ uncorrelated indices that keep most of the variance. Used for *visualising*, *compressing*, *de-noising*, and as inputs to a later supervised model (principal-components regression).

Clustering — K-means, hierarchical

Partition the n observations into groups that are similar within and different between. Used for *segmentation*, *subtype discovery*, *triage*, and generating hypotheses to test later.

Run this live in the lab notebook

The companion notebook `advanced_08_unsupervised_lab.ipynb` runs every method in this chapter on `USArrests` and on the bundled gene-expression data.

Exercise 12.1 — Why there is no test error [Concept]

Exercise

A colleague fits K-means with $K = 5$ to customer data and says: "I held out 20% of the customers, clustered the other 80%, assigned the held-out ones to their nearest centroid, and the assignment error was zero — so the clustering is validated."

1. Explain precisely what is wrong with that argument.
2. Name one quantity you *can* compute from held-out data that is genuinely informative about a clustering, and say what it does and does not establish.
3. Give one way to obtain real external evidence that a segmentation is useful.

Solution — Exercise 12.1

Solution

Part 1 — the circularity. "Assign each held-out point to its nearest centroid" *defines* the label; there is no independent label to disagree with it, so the "error" is zero by construction and carries no information. A test set works in Chapter 2 only because the held-out y_i was fixed *before* the model existed.

Part 2 — what you can compute: stability. Cluster the two halves separately, match the two solutions and measure agreement (e.g. the adjusted Rand index), or bootstrap and record how often two customers land together. High stability shows the partition is *reproducible*; it does **not** show it is real. Pure noise clusters very stably — the axes of the split are determined by the noise's own geometry.

Part 3 — external evidence. Convert it into a supervised question: run a campaign on two segments with matched controls and measure uplift; or check whether segment membership improves prediction of an outcome that was never used to build it (churn, default, response rate).

Common mistake

Calling within-cluster sum of squares an "error". It is the *objective* the algorithm minimises, so it always improves with more clusters — comparing it across K is not validation, it is bookkeeping.

Outline

1 No Response
2 PCA
3 K-means
4 Hierarchical
5 Practice
6 Python Lab
7 Summary

The problem PCA solves

With p features there are $\binom{p}{2}$ scatterplots. For $p = 10$ that is 45 pictures; for $p = 1000$ genes it is 499 500.

The idea

Most of those plots are nearly redundant, because the features are correlated. PCA looks for a *few* directions in feature space along which the data actually varies, and plots those instead.

USArrests, $p = 4$

Correlations: Murder–Assault 0.80, Murder–Rape 0.56, Assault–Rape 0.67, UrbanPop–Rape 0.41, Murder–UrbanPop 0.07. Three of the four variables move together; a single "violent crime" index should capture most of the picture.

History: PCA is one of the oldest tools in this course — Karl Pearson (1901), reinvented and named by Harold Hotelling (1933).

The first principal component: the direction of maximum variance

Let the columns of X be centred. The first component is the normalised linear combination with the largest sample variance:

Rule

$$\max_{\phi_{11},\ldots,\phi_{p1}} \frac{1}{n}\sum_{i=1}^{n}\Big(\sum_{j=1}^{p}\phi_{j1}x_{ij}\Big)^2 \quad \text{subject to} \quad \sum_{j=1}^{p}\phi_{j1}^2 = 1.$$

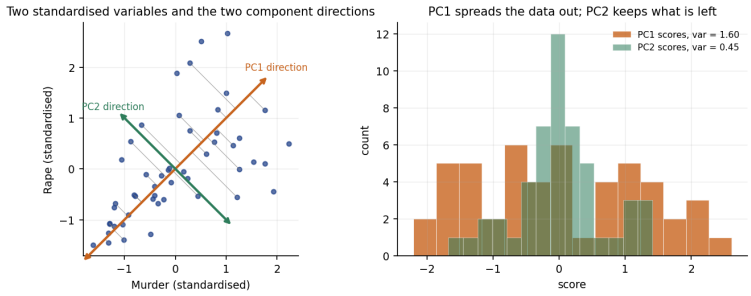
How to read this

- ϕ_{j1} — the **loading** of feature j: how much of feature j goes into the component. The vector ϕ_1 is a *direction*.
- $z_{i1} = \sum_j \phi_{j1}x_{ij}$ — the **score** of observation i: its coordinate along that direction.
- The constraint $\|\phi_1\| = 1$ is what stops the "variance" being inflated by simply making the loadings large.
- Centring is required; the maximised quantity is a variance only if the scores have mean 0.

Takeaway

The solution is the eigenvector of the covariance (or correlation) matrix belonging to its largest eigenvalue λ_1, and λ_1 *is* the variance of the scores z_{i1}.

Seeing it in two dimensions



Takeaway

Murder and Rape (standardised, $r = 0.564$) are stretched along the 45° line. PC1 points there and its scores have variance 1.60; PC2 is forced perpendicular and gets only 0.45. The grey stubs are the residuals PC1 *discards* — the shortest possible ones, which is the same thing as the widest possible spread.

Two variables by hand: the whole of PCA in one calculation

Murder and Rape, standardised, $r = 0.564$

The correlation matrix is $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$. Its eigenvectors are always $\frac{1}{\sqrt{2}}(1,1)^\top$ and $\frac{1}{\sqrt{2}}(1,-1)^\top$, with eigenvalues

$$\lambda_1 = 1 + r = 1.564, \qquad \lambda_2 = 1 - r = 0.436, \qquad \lambda_1 + \lambda_2 = 2 = p.$$

So $\mathrm{PVE}_1 = \frac{1+r}{2} = 78.2\%$ and $\mathrm{PVE}_2 = \frac{1-r}{2} = 21.8\%$. (The previous slide's 1.60 and 0.45 are the same two eigenvalues computed with the $n-1$ divisor: $\frac{50}{49} \times 1.564 = 1.60$. The ratio, and so the PVE, is identical.)

Takeaway

Every fact about PCA is visible here: the loadings are a *direction* $(0.707, 0.707)$, the components are orthogonal, the eigenvalues sum to the total variance, and the more correlated the features, the more the first component captures. Uncorrelated features ($r = 0$) give $\lambda_1 = \lambda_2 = 1$ — PCA has nothing to find.

Later components, and why they are orthogonal

Definition

ϕ_2 maximises the variance of $z_{i2} = \sum_j \phi_{j2} x_{ij}$ subject to $\|\phi_2\| = 1$ *and* $\phi_2^\top \phi_1 = 0$; and so on for $m = 3, \ldots, p$. Consequently

$$\mathrm{Cov}(z_{\cdot 1}, z_{\cdot 2}) = 0, \qquad \sum_{m=1}^{p} \lambda_m = \sum_{j=1}^{p} \mathrm{Var}(X_j).$$

How to read this

- Orthogonality is imposed as a *constraint*, not discovered: it is what makes each component report *new* variance instead of repeating PC1.
- Because the score vectors are uncorrelated, PCA *partitions* the total variance — the λ_m add up, with no double counting. There are at most $\min(n-1, p)$ of them with non-zero variance.

Takeaway

PCA is a change of axes, not a model: no assumption, no error term, nothing fitted — which is also why nothing can be tested.

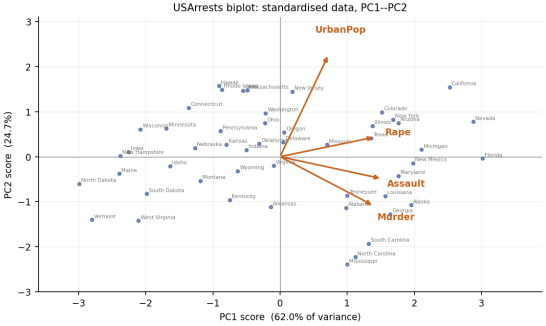
USArrests: the loadings, read as sentences

	PC1	PC2	PC3	PC4
Murder	0.536	−0.418	−0.341	−0.649
Assault	0.583	−0.188	−0.268	0.743
UrbanPop	0.278	0.873	−0.378	−0.134
Rape	0.543	0.167	0.818	−0.089
λ_m (variance)	2.480	0.990	0.357	0.173

Reading the first two columns

PC1 loads on Murder, Assault and Rape at roughly equal weight and on UrbanPop only half as much: an overall *serious-crime rate*. **PC2** is almost entirely UrbanPop (0.873), the crime variables pulling the other way: *urbanisation at a given crime level*.

The biplot: scores and loadings in one picture



Takeaway

Florida (PC1 = 3.01) and Nevada sit at the crime end; North Dakota (−2.99) and Vermont at the quiet end. Hawaii and California are both high on PC2 (urban); Mississippi is lowest (−2.39) — high crime, rural.

How to read a biplot — and how not to

Four reading rules

1. **Points** are observations in the plane of the first two scores; nearby points have similar profiles *as far as PC1 and PC2 go*.
2. **Arrows** are the loadings: an arrow pointing right means high values of that feature push a state to the right.
3. **Angles between arrows** approximate correlations — small angle, positively correlated; right angle, roughly uncorrelated (UrbanPop vs Murder, $r = 0.07$).
4. **Arrow length** indicates how well the feature is represented by these two components, not how important it is.

Common mistake

Reading distances between distant points as if the plot were the data. It is a *projection*: here it keeps 86.8% of the variance, so 13.2% of the variation is not in the picture at all, and two points that look adjacent may differ sharply on PC3.

Signs are arbitrary — your plot may be a mirror image

The indeterminacy

If ϕ_m maximises the variance, so does $-\phi_m$: it is the same direction, and $\|-\phi_m\| = 1$ too. Flipping it flips every score on that component:

$$\phi_m \mapsto -\phi_m \quad \Longrightarrow \quad z_{im} \mapsto -z_{im} \quad \text{for all } i, \qquad \lambda_m \text{ unchanged.}$$

The concrete case in this chapter

`scikit-learn` returns PC2 loadings $(-0.418, -0.188, 0.873, 0.167)$ for `USArrests`. ISLP Figure 12.1 prints $(0.418, 0.188, -0.873, -0.167)$. Both are correct, and the two biplots are vertical mirror images of each other.

Common mistake

Concluding you have made an error because your figure is upside down relative to the book. Check the *eigenvalues* and the *absolute* loadings — those are unique. If you must fix a sign for reproducibility, adopt a convention (e.g. force the largest-magnitude loading of each component to be positive) and state it.

Proportion of variance explained

Definition

$$\mathrm{PVE}_m = \frac{\lambda_m}{\sum_{m'=1}^{p} \lambda_{m'}} = \frac{\sum_{i=1}^{n} z_{im}^2}{\sum_{j=1}^{p} \sum_{i=1}^{n} x_{ij}^2}, \qquad \sum_{m=1}^{p} \mathrm{PVE}_m = 1.$$

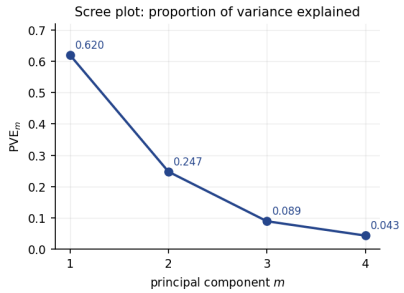
How to read this

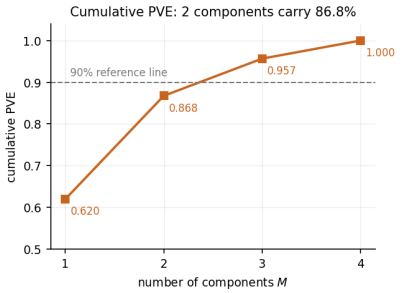
- Numerator: variation the m-th component accounts for. Denominator: total variation in the (centred) data. On *standardised* data the denominator is exactly np and $\sum_m \lambda_m = p$ — for `USArrests`, λ sums to 4 and the total sum of squares is $50 \times 4 = 200$.
- Cumulative PVE $= \sum_{m \le M} \mathrm{PVE}_m$ is what you report when you keep M components. Its complement $1 - \sum_{m \le M} \mathrm{PVE}_m$ is exactly the fraction of squared error left when the data is rebuilt from those components, because PCA *is* the best rank-M approximation (*appendix*).

Takeaway

PVE is a descriptive statistic of *this* data set: not an out-of-sample quantity, and never a statement that the components are "right".

The scree plot — and why choosing M is a judgement call





Takeaway

$\mathrm{PVE} = (0.620, 0.247, 0.089, 0.043)$: two components carry 86.8%, so a two-dimensional picture of the 50 states is defensible. The "elbow" at $m = 2$ is a kink in a curve, not a test — there is no held-out quantity that turns $M = 2$ into the right answer, and $M = 3$ (95.7%) is equally defensible.

Exercise 12.2 — PCA from a 2×2 correlation matrix [Math]

Exercise

Two standardised features have correlation $r = 0.80$ (this is the real Murder–Assault correlation in `USArrests`, to two decimals), so their correlation matrix is

$$\mathrm{R} = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}.$$

1. Find both eigenvalues of R and the corresponding normalised loading vectors.
2. Compute PVE_1 and PVE_2.
3. An observation has standardised values $(x_1, x_2) = (1.5, 1.0)$. Compute its two scores.
4. State what changes if you replace ϕ_1 by $-\phi_1$.

Solution — Exercise 12.2

Solution

Part 1 — eigen-decomposition. For $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$, $\det(\mathrm{R} - \lambda I) = (1-\lambda)^2 - r^2 = 0$, so $1 - \lambda = \pm r$ and

$$\lambda_1 = 1 + r = 1.8, \qquad \lambda_2 = 1 - r = 0.2.$$

Substituting λ_1 gives $-0.8\phi_1 + 0.8\phi_2 = 0$, i.e. $\phi_1 = \phi_2$; normalising,

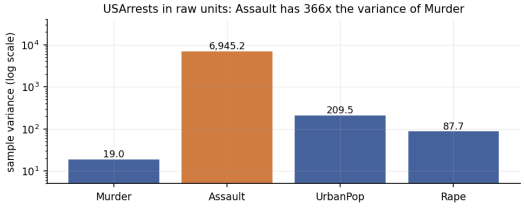
$$\phi_1 = \tfrac{1}{\sqrt{2}}(1,1)^\top = (0.707,\, 0.707)^\top, \qquad \phi_2 = \tfrac{1}{\sqrt{2}}(1,-1)^\top = (0.707,\, -0.707)^\top.$$

Part 2 — PVE. Total variance $= \lambda_1 + \lambda_2 = 2 = p$, so $\mathrm{PVE}_1 = 1.8/2 = 90\%$ and $\mathrm{PVE}_2 = 0.2/2 = 10\%$.

Part 3 — scores. $z_1 = 0.707(1.5) + 0.707(1.0) = 0.707(2.5) = 1.768$ and $z_2 = 0.707(1.5) - 0.707(1.0) = 0.707(0.5) = 0.354$. This state is well above average on both features, so it is far out on the "overall level" component and only slightly off the "difference" component.

Part 4 — sign flip. λ_1, PVE_1 and all distances are unchanged; every score on PC1 changes sign, so $z_1 = -1.768$. The picture is mirrored, the analysis is identical.

Scaling is not optional — look at the raw variances



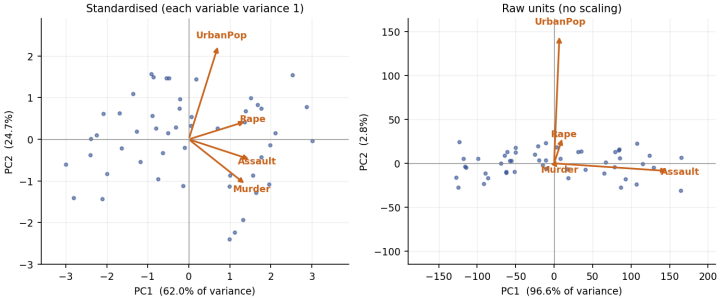
Why PCA cannot ignore this

`Assault` is counted per 100 000 and `Murder` is too, but murders are rarer: variances are 6945.2 and 19.0, a factor of 366. PCA maximises variance, so on raw data the first component has almost no choice but to be Assault.

Takeaway

Recording `Assault` per million instead of per 100 000 would multiply its variance by 100 and change the answer again. PCA on unstandardised data is a statement about *units*, not about the world.

The same data, standardised and not



Takeaway

Left: PC1 takes 62.0% and loads on all four variables $(0.54, 0.58, 0.28, 0.54)$. Right: PC1 takes 96.6% and is Assault alone (loading 0.995; Murder gets 0.042), PC2 is UrbanPop, and 99.3% of "the structure" is two variables' measurement scales. The right-hand panel is not wrong arithmetic — it is the answer to a different, worse question.

When to standardise, and when not to

The rule

- Features in **different units** (euros, years, counts, percentages) ⇒ standardise. This is the default, and it means PCA on the *correlation* matrix.
- Features in the **same unit and genuinely comparable** — gene expression on one platform, the same price in 30 maturities, pixels ⇒ centring alone may be right, because a channel that varies more really is more informative.
- Never scale a variable up because it "matters more". That is a modelling choice being smuggled in as a preprocessing step.

Common mistake

Standardising *after* splitting, or with statistics computed on the full data before a split. Fit the scaler where you fit the model — the Chapter 5 leakage rule applies unchanged.

Run this live in the lab notebook

§1 of the companion notebook runs both versions side by side and prints both PVE vectors and PC1 loadings.

Exercise 12.3 — Scaled vs unscaled PCA on USArrests [Python]

Exercise

Using `USArrests` ($n = 50$, $p = 4$):

1. Run PCA on the **standardised** data. Report the four PVEs and the PC1 loadings.
2. Run PCA on the **centred but unscaled** data. Report the same two things.
3. Explain the difference by reference to the raw variances, and state which analysis you would put in a report.

Run this live

The worked solution sits in `advanced_08_unsupervised_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Exercise 12.3 — Scaled vs unscaled PCA on USArrests [Python]*.

Solution — Exercise 12.3 (1/2)

Solution — code

```
import numpy as np, pandas as pd # arrays and frames
from sklearn.decomposition import PCA # the PCA estimator
from sklearn.preprocessing import StandardScaler # mean 0, sd 1 per column

USA = load('USArrests', index_col=0) # 50 states x 4 features
print(USA.var().round(1).to_dict()) # Assault 6945, Murder 19

for tag, X in [('scaled', StandardScaler().fit_transform(USA)),
               ('unscaled', (USA - USA.mean()).values)]: # centred, NOT scaled
    p = PCA().fit(X) # same estimator both ways
    print(tag, 'PVE', np.round(p.explained_variance_ratio_, 4))
    print(tag, 'PC1', dict(zip(USA.columns, p.components_[0].round(3))))
```

How to read this

One loop, one estimator, two preprocessings — so any difference in the printout is the scaling and nothing else.

Solution — Exercise 12.3 (2/2)

Solution — output and interpretation

Raw variances. Murder 19.0, Assault 6945.2, UrbanPop 209.5, Rape 87.7.

	PVE_1	PVE_2	PVE_3	PVE_4	PC1 loadings (Mur, Ass, Urb, Rape)
scaled	0.6201	0.2474	0.0891	0.0434	(0.536, 0.583, 0.278, 0.543)
unscaled	0.9655	0.0278	0.0058	0.0009	(0.042, 0.995, 0.046, 0.075)

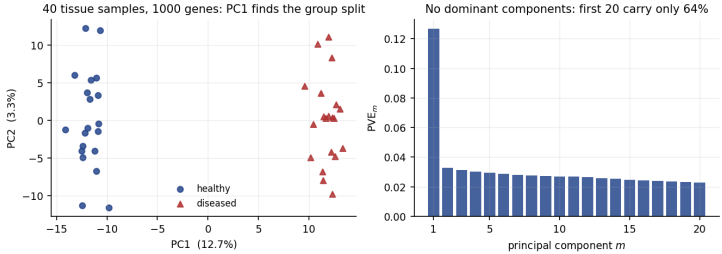
Part 3 — why. PCA maximises variance, and `Assault` carries 366 times the variance of `Murder` purely because assaults are 22 times more common. The unscaled PC1 is therefore `Assault` with a rounding error attached, and its 96.6% measures the dominance of one column's units, not structure.

What goes in the report. The **standardised** analysis — the four variables are on different scales and no one of them deserves 366 times the weight. Say so explicitly in the methods section.

Common mistake

Reporting a triumphant "PC1 explains 96.6% of the variance". That is the largest number available and the least informative one.

PCA when $p \gg n$: 1000 genes, 40 samples



Takeaway

Here PC1 explains only 12.7% — and separates the 20 healthy from the 20 diseased samples completely (group means -11.8 and $+11.8$). A small PVE is not a failure: with $p \gg n$ the variance is spread over 39 components (the first 20 hold 64%), and the interesting direction need not be the biggest one.

Live demo — §4 of the notebook

Switch to `advanced_08_unsupervised_lab.ipynb` §4: $\mathrm{PVE}_1 = 0.127$, PC1 group means ± 11.75.

Outline

1 No Response

2 PCA

3 K-means

4 Hierarchical

5 Practice

6 Python Lab

7 Summary

Clustering: what we are asking for

The requirement

A partition $C_1, \ldots, C_K$ of the n observations with

$$C_1 \cup \cdots \cup C_K = \{1, \ldots, n\}, \qquad C_k \cap C_{k'} = \varnothing \quad (k \neq k'),$$

such that observations inside a cluster are similar and observations in different clusters are not.

Two designs, two commitments

- K-**means** — you must fix K first; you get one flat partition.
- **Hierarchical** — you fix nothing first; you get a tree and choose K afterwards by cutting it.

Neither is a model of how the data arose, so neither can be checked against the data. And neither chooses the features or the distance for you: grouping insurance claims by cost alone puts a cheap fraud next to a cheap accident. Choosing what "similar" means *is* the analysis; the algorithm is the easy part.

The K-means objective

Rule — minimise total within-cluster variation

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} W(C_k), \quad W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2$$

How to read this

- $W(C_k)$ sums the squared Euclidean distances over all pairs inside cluster k and divides by $|C_k|$ — small means tight.
- The second equality is an algebraic identity (derived in the appendix): the *centroid form*, with $\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$. Software reports it without the factor 2 — `sklearn`'s `inertia_`.
- Only the *partition* is optimised; the centroids are a consequence.

Takeaway

Because the criterion is squared Euclidean distance, K-means implicitly wants **round, equally sized, equally spread** clusters — and will impose them whether or not the data has them.

The algorithm, step by step

Lloyd's algorithm

1. Assign each observation at random to one of K clusters (or place K initial centroids).
2. Compute the K centroids $\bar{x}_k$ of the current clusters.
3. Re-assign every observation to the cluster with the nearest centroid, in squared Euclidean distance.
4. Repeat 2–3 until no assignment changes.

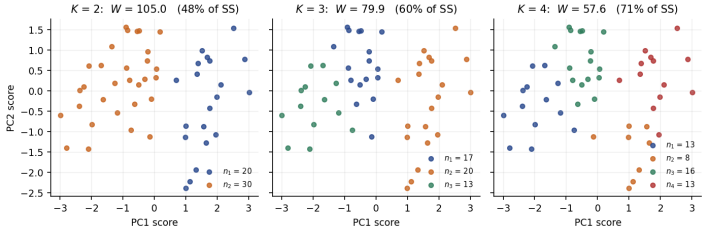
Why it stops, and what it stops at

Step 2 cannot increase $\sum_k W(C_k)$ (the mean minimises squared distance) and neither can step 3 (each point moves only to a closer centroid). The objective therefore decreases monotonically, there are finitely many partitions, so the algorithm **must terminate** — at a partition no single move can improve. That is a *local* optimum.

Takeaway

Guaranteed to converge, not guaranteed to be right. There are $\sim K^n/K!$ partitions; the algorithm visits a handful.

USArrests at $K = 2$, 3 and 4



Takeaway

Total sum of squares is 200 (standardised, 50×4). W falls $200 \to 105.0 \to 79.9 \to 57.6$, i.e. 47%, 60%, 71% "explained". It will keep falling to 0 at $K = 50$, so this sequence contains no evidence about which K is right — plotted on PC1–PC2, the clusters are mostly slices of PC1.

Live demo — §2 of the notebook

Run the K-loop cell: $W = 200.0,\ 105.0,\ 79.9,\ 57.6$.

History: the algorithm is Stuart Lloyd's (Bell Labs, 1957), unpublished until 1982; the name "K-means" is MacQueen's (1967).

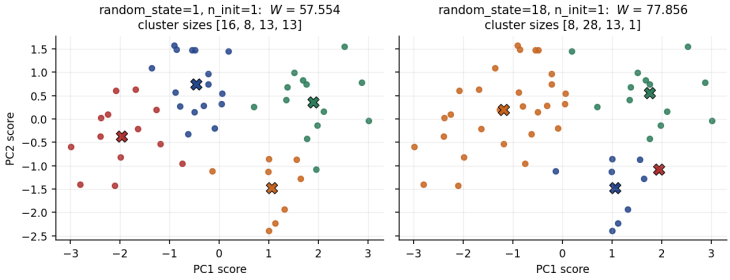
Are the clusters interpretable? The $K = 3$ profiles

	Murder	Assault	UrbanPop	Rape	
Cluster 1 ($n = 20$)	12.2	255.2	68.4	29.2	high crime, urban
Cluster 2 ($n = 17$)	5.8	141.9	72.5	18.8	moderate crime, most urban
Cluster 3 ($n = 13$)	3.6	78.5	52.1	12.2	low crime, rural
Overall	7.8	170.8	65.5	21.2	

The only honest check available on this slide

Cluster means in the *original units* are how a clustering earns its keep: if the three rows read as three recognisable kinds of state, the partition is useful. That is interpretability, not validation — three groups ordered along one crime axis is exactly what K-means would produce from a single unimodal cloud too. Note also that a table like this one, and not the vector of labels, is what anyone downstream actually uses: nothing can be written against a cluster number.

K-means finds a local optimum — with real numbers



Takeaway

Same data, same $K = 4$, same code — only the random start differs (`n_init=1`). Seed 1 reaches $W = 57.554$ with sizes $(16, 8, 13, 13)$; seed 18 stops at $W = 77.856$ with sizes $(8, 28, 13, 1)$, having merged two groups and left one state alone. Fifteen of the fifty states change cluster (ARI 0.573), and neither run knows the other exists.

The practical defence: many starts, and say how many

Twenty single-start runs, $K = 4$, `USArrests` standardised

Seeds 0–19 give, sorted: 57.554 (×7), 57.668 (×2), 57.673, 58.207 (×3), 58.242 (×3), 58.782, 58.962, 71.875, 77.856 — **nine** distinct local optima. Thirteen of the twenty runs miss the best value and the worst is 35% above it; with `n_init=50`, $W = 57.554$ every time.

What to do

- **Always set `n_init` explicitly**, to 20–50. Since `scikit-learn` 1.4 the default is `n_init='auto'`, which with the default `k-means++` initialisation means **one start**: `KMeans(n_clusters=4, random_state=0)` on this data returns $W = 58.242$, not the optimum 57.554.
- Report the objective, `n_init` and the seed — a clustering without them is not reproducible. If several starts give *equally good* objectives but different partitions, the structure is weak, and that is information too.

Common mistake

Treating K-means as deterministic and comparing two teams' cluster 3: even at the same optimum the *labels* are arbitrary.

Exercise 12.4 — Two local optima on four points [Math]

Exercise

Four observations sit at the corners of a rectangle:

$$x_1 = (0,0), \quad x_2 = (0,1), \quad x_3 = (4,0), \quad x_4 = (4,1),$$

and we run K-means with $K = 2$. Use $W = \sum_k \sum_{i \in C_k} \|x_i - \bar{x}_k\|^2$ (`sklearn`'s `inertia_`).

1. Compute W for the *vertical* split $\{x_1, x_2\}, \{x_3, x_4\}$.
2. Compute W for the *horizontal* split $\{x_1, x_3\}, \{x_2, x_4\}$.
3. Show that the horizontal split is a fixed point of the algorithm — no single observation wants to move.
4. What does this imply about running K-means once?

Solution — Exercise 12.4 (1/3)

Solution

Part 1 — vertical split. Centroids $(0, 0.5)$ and $(4, 0.5)$. Each point is 0.5 from its centroid, so

$$W = 4 \times (0.5)^2 = 1.0.$$

Part 2 — horizontal split. Centroids $(2, 0)$ and $(2, 1)$. Each point is 2 away in x_1 and 0 in x_2:

Solution — Exercise 12.4 (2/3)

Solution

$$W = 4 \times 2^2 = 16.0.$$

Part 3 — it is stable. Take $x_1 = (0, 0)$ under the horizontal split. Squared distances to the two centroids are

$$\|x_1 - (2, 0)\|^2 = 4 \qquad \text{and} \qquad \|x_1 - (2, 1)\|^2 = 4 + 1 = 5.$$

It is already in the nearer cluster, so it stays; by symmetry so does every other point. The centroids do not move either, so step 3 changes nothing and the algorithm **terminates here**, reporting $W = 16$ — sixteen times the optimum.

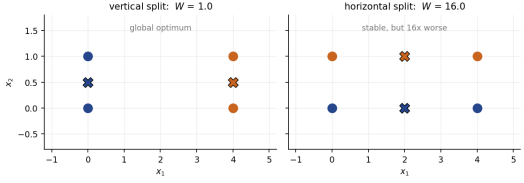
Solution — Exercise 12.4 (3/3)

Solution

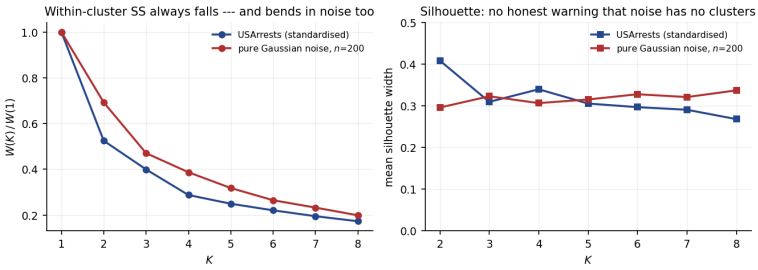
Part 4 — consequence. A single run of K-means can return an arbitrarily bad partition and give no sign of it: the algorithm's stopping rule ("no point wants to move") is satisfied at both $W = 1$ and $W = 16$. Only comparing *several* random starts and keeping the lowest W protects you, which is what `n_init` does.

Verification in code. Initialising at $(0, 0.5), (4, 0.5)$ gives $W = 1.0$; at $(2, 0), (2, 1)$ it gives $W = 16.0$; `n_init`=20 returns 1.0.

Common mistake: concluding that a *converged* run is a *good* run. Convergence is a fact about the algorithm; quality is about W, and W is only interpretable relative to other starts at the *same* K.



Choosing K: what the standard diagnostics can and cannot do



Takeaway

Left: $W(K)/W(1)$ falls smoothly for `USArrests` *and* for 200 points of pure Gaussian noise — the noise curve bends just as convincingly (0.69, 0.47, 0.39 at $K = 2, 3, 4$). Right: the mean silhouette on noise sits at 0.30–0.34 for every K and is *highest at* $K = 8$. These diagnostics rank candidate K; they never say "there are no clusters".

Extended Exercise 12.1 — How bad is the local-optimum problem? [Python]

Extended exercise (15 min)

On standardised `USArrests`, quantify the local-optimum problem instead of asserting it.

1. For $K = 4$, run K-means with `n_init=1` for seeds $0, \ldots, 19$ and collect the objective `inertia_` and the sorted cluster sizes.
2. Report how many *distinct* local optima appear, the best and worst objective, and the percentage gap between them.
3. Compare with a single run at `n_init=50`, and repeat the whole exercise at $K = 8$.
4. State the reporting rule you would impose on a team from these numbers.

Run this live

The worked solution sits in `advanced_08_unsupervised_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Extended Exercise 12.1 — How bad is the local-optimum problem? [Python]*.

Solution — Extended Exercise 12.1 (1/2)

Solution — code

```
from sklearn.cluster import KMeans # Lloyd's algorithm
from sklearn.preprocessing import StandardScaler # W depends on the scaling

Xs = StandardScaler().fit_transform(load('USArrests', index_col=0))

for K in (4, 8): # rougher surface at K=8
    runs = [KMeans(n_clusters=K, n_init=1, random_state=s).fit(Xs)
            for s in range(20)] # 20 seeds, one start each
    w = np.array([r.inertia_ for r in runs]) # the objective, once per seed
    best = KMeans(n_clusters=K, n_init=50, random_state=0).fit(Xs).inertia_
    print(f'K={K}: {len(np.unique(w.round(3)))} optima min={w.min():.3f} '
          f'max={w.max():.3f} gap={100*(w.max()/w.min()-1):.1f}% '
          f'n_init=50 -> {best:.3f}') # the value 50 starts reach
    print(' best / worst sizes', sorted(np.bincount(runs[w.argmin()].labels_)),
          sorted(np.bincount(runs[w.argmax()].labels_))) # a bad W is a bad split
```

Solution — Extended Exercise 12.1 (2/2)

Solution — output and the reporting rule

Printout.

```
K=4:  9 optima  min=57.554  max=77.856  gap=35.3%  n_init=50 -> 57.554
          best / worst sizes [8, 13, 13, 16] [1, 8, 13, 28]
K=8:  20 optima  min=36.551  max=40.965  gap=12.1%  n_init=50 -> 34.636
```

- **Parts 1–2.** At $K = 4$, twenty starts land on *nine* different local optima and the worst is 35.3% above the best. The failure is structural, not a rounding issue: the worst run returns sizes $(1, 8, 13, 28)$ — one state alone in its own cluster and more than half in another — against $(8, 13, 13, 16)$ for the best.
- **Part 3.** At $K = 8$ all twenty starts give twenty *different* values (36.551 to 40.965) and *none* of them reaches the `n_init`=50 value of 34.636. More clusters means a rougher objective surface and more places to get stuck — exactly where a single run is least trustworthy.
- **Part 4 — the rule.** Every reported clustering states K, the dissimilarity, the scaling, `n_init` (≥ 20), the seed and the achieved objective. Two runs whose W differs are not two opinions; one of them is simply worse.

Outline

1 No Response
2 PCA
3 K-means
4 Hierarchical
5 Practice
6 Python Lab
7 Summary

Hierarchical clustering: commit to nothing in advance

What it gives you that K-means does not

- No K is chosen before fitting. One tree contains the partition for *every* K from 1 to n.
- The partitions are **nested**: the 3-cluster solution always refines the 2-cluster one.
- It needs only a table of pairwise dissimilarities, so it works where no "mean" exists (text, categorical profiles, correlations).
- But nesting is a real *restriction*: if the best two-group split of your customers is by country and the best three-group split is by product, no dendrogram shows both.

Bottom-up (agglomerative) algorithm

1. Start with n clusters of one observation each and all $\binom{n}{2}$ pairwise dissimilarities.
2. Fuse the two *least dissimilar* clusters; record the dissimilarity as the **height** of that fusion.
3. Recompute dissimilarities between the new cluster and the rest using the chosen **linkage**; repeat until one cluster remains.

Reading a dendrogram

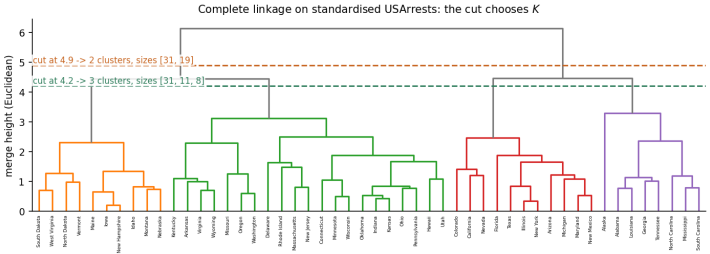
Three rules, in order of importance

1. **Height is everything.** Two observations are similar if the height at which their branches *first join* is low. Read the vertical axis.
2. **Horizontal position means nothing.** A dendrogram has 2^{n-1} equally valid drawings; the ordering of leaves is a plotting choice.
3. **Cutting** at a height gives a partition; the number of vertical lines the cut crosses is K.

Takeaway

In the complete-linkage tree overleaf Alaska is drawn immediately next to Alabama, yet the two only join at height 3.29; Alabama and *Louisiana* join at 0.78, and Alabama and Iowa not until 6.14. Adjacency lied; the heights did not.

Cutting the tree chooses K — after you have seen it



Takeaway

Complete linkage on standardised `USArrests`: cutting at 4.9 gives 2 clusters of sizes $(31, 19)$; cutting at 4.2 gives 3 of sizes $(31, 11, 8)$. Both cuts are legitimate. Choosing the height *after* looking at the tree is exactly the selection problem module A7 formalises — and here there is no p-value to correct.

Live demo — §3 of the notebook

Switch to `advanced_08_unsupervised_lab.ipynb` §3 and cut the tree: sizes $(31, 19)$ and $(31, 11, 8)$.

Exercise 12.5 — Reading a dendrogram [Concept]

Exercise

A dendrogram over five observations has fusion heights: $\{A, B\}$ at 0.4; $\{C, D\}$ at 0.6; $\{A, B\}$ with E at 1.5; $\{A, B, E\}$ with $\{C, D\}$ at 2.8.

1. Which pair of observations is the most similar, and which observation is the last individual leaf to join a cluster?
2. Give the partitions obtained by cutting at heights 0.5, 1.0, 2.0 and 3.0.
3. In one drawing E appears immediately to the left of C. What, if anything, does that tell you about E and C?
4. Why is "cut wherever the biggest vertical gap is" not a validation procedure?

Solution — Exercise 12.5 (1/2)

Solution

Part 1. A and B fuse lowest (0.4), so they are the most similar pair. The last individual leaf to join anything is E, at height 1.5 — compared with 0.4 for A, B and 0.6 for C, D, so E is the most isolated single observation. (The fusion at 2.8 joins two *clusters*, not a leaf.)

Solution — Exercise 12.5 (2/2)

Solution

Part 2 — cuts.

height	partition	K
0.5	$\{A, B\}, \{C\}, \{D\}, \{E\}$	4
1.0	$\{A, B\}, \{C, D\}, \{E\}$	3
2.0	$\{A, B, E\}, \{C, D\}$	2
3.0	$\{A, B, C, D, E\}$	1

Part 3. Nothing at all. Horizontal adjacency is an artefact of the drawing; E and C first share a cluster only at height 2.8, the largest fusion in the tree, so they are in fact among the *least* similar pairs.

Part 4. The gap is computed from the same data that produced the tree, so it cannot be contradicted by it: any data set, structured or not, has a largest gap. It is a reasonable heuristic for *proposing* K and no evidence that K clusters exist.

Linkage: how to measure the distance between two *groups*

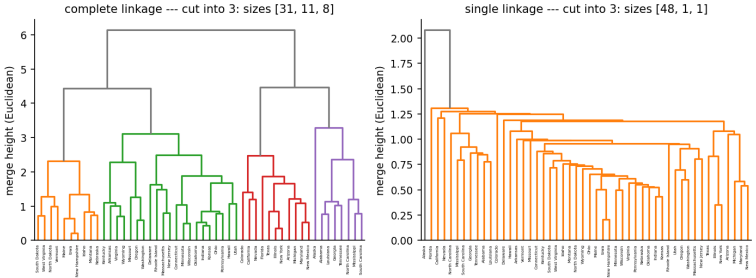
The four classical choices, for groups A and B

Complete	$d(A,B) = \max_{i\in A,\, i'\in B} d(i,i')$
Single	$d(A,B) = \min_{i\in A,\, i'\in B} d(i,i')$
Average	$d(A,B) = \frac{1}{\lvert A\rvert\lvert B\rvert} \sum_{i\in A}\sum_{i'\in B} d(i,i')$
Centroid	$d(A,B) = \lVert \bar{x}_A - \bar{x}_B \rVert^2$

What each one does to the shape of the answer

- **Complete** — a cluster is only as good as its worst pair: compact, similarly sized, balanced clusters.
- **Single** — one close pair is enough to fuse: produces long chains and strips off singletons (*chaining*).
- **Average** — the compromise; with complete, the usual default.
- **Centroid** — used in genomics, but can produce an **inversion** (a fusion *lower* than one of its children), which makes the picture hard to read.

One data set, two linkages, two different answers



Takeaway

Identical data, identical Euclidean distances, identical cut into 3. Complete linkage returns balanced clusters of sizes $(31, 11, 8)$; single linkage returns $(48, 1, 1)$ — it peels off Alaska and Florida one at a time and calls the other 48 states one cluster. The agreement between the two partitions is an adjusted Rand index of 0.079: essentially none.

How much does the linkage choice matter? Quantify it

Standardised `USArrests`, Euclidean, cut into $K = 3$

linkage	sizes	tallest fusion	ARI vs complete
complete	$(31, 11, 8)$	6.14	1.000
average	$(30, 19, 1)$	3.36	0.784
single	$(48, 1, 1)$	2.08	0.079
Ward	$(19, 12, 19)$	13.65	0.465

Centroid linkage reproduces average exactly here. K-means at $K = 3$ agrees with complete at ARI 0.394, average 0.607, Ward 0.878 — and single -0.013, worse than chance.

Takeaway

Five defensible methods, five different partitions of the same 50 states, no quantity in the data ranking them. The linkage is a **modelling assumption about cluster shape**: choose it before you see the answer, and say which you chose — the distance, the linkage and the cut height, all three, so that a reader can reproduce the partition.

Exercise 12.6 — Complete vs single linkage by hand [Math]

Exercise

Four observations have the dissimilarity matrix

$$D = \begin{pmatrix} 0 & 0.30 & 0.40 & 0.70 \\ 0.30 & 0 & 0.50 & 0.80 \\ 0.40 & 0.50 & 0 & 0.45 \\ 0.70 & 0.80 & 0.45 & 0 \end{pmatrix}.$$

1. Build the dendrogram under **complete** linkage, giving every fusion height.
2. Build it under **single** linkage.
3. In each case, which clusters result from cutting the tree so that two clusters remain?
4. Explain the difference in terms of the definitions.

Solution — Exercise 12.6 (1/2)

Solution — complete linkage

Step 1. The smallest entry is $d(1,2) = 0.30$: fuse $\{1,2\}$ at height 0.30.

Step 2 — update with the maximum.

$$d(\{1,2\},3) = \max(0.40, 0.50) = 0.50, \qquad d(\{1,2\},4) = \max(0.70, 0.80) = 0.80, \qquad d(3,4) =$$

The smallest is now 0.45: fuse $\{3,4\}$ at height 0.45.

Step 3. $d(\{1,2\},\{3,4\}) = \max(0.40, 0.50, 0.70, 0.80) = 0.80$: the final fusion is at height 0.80.

Tree. $\{1,2\}$ at 0.30, $\{3,4\}$ at 0.45, root at 0.80. Cutting below 0.80 leaves **two clusters: $\{1,2\}$ and $\{3,4\}$.**

Solution — Exercise 12.6 (2/2)

Solution — single linkage, and the comparison

Step 1. Same first fusion: $\{1, 2\}$ at height 0.30.

Step 2 — update with the minimum.

$$d(\{1,2\},3) = \min(0.40, 0.50) = 0.40, \qquad d(\{1,2\},4) = \min(0.70, 0.80) = 0.70, \qquad d(3,4) = 0.45.$$

Now the smallest is 0.40, so observation 3 joins $\{1, 2\}$ at height 0.40 — *before* 3 and 4 ever meet.

Step 3. $d(\{1,2,3\},4) = \min(0.70, 0.80, 0.45) = 0.45$: final fusion at 0.45. Cutting below it leaves $\{1, 2, 3\}$ **and** $\{4\}$**.**

Part 4 — why. Complete linkage judged $\{1, 2\}$ and 3 by their *worst* pair (0.50), which let 3 and 4 pair up first. Single linkage judged them by their *best* pair (0.40) and chained 3 onto $\{1, 2\}$, leaving 4 alone. Same distances, incompatible answers — and single linkage's heights are systematically lower, so its tree looks flatter.

Common mistake: recomputing group distances from the *original* matrix each round, or averaging the fused rows when you meant to take a maximum.

The dissimilarity measure: often the bigger decision

Two common choices

$$\underbrace{d(i,i') = \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2}_{\text{squared Euclidean: cares about } \textit{level}} \qquad \underbrace{d(i,i') = 1 - r_{ii'}}_{\text{correlation-based: cares about } \textit{shape}}$$

where $r_{ii'}$ is the correlation across the p features between observation i's and i''s profiles.

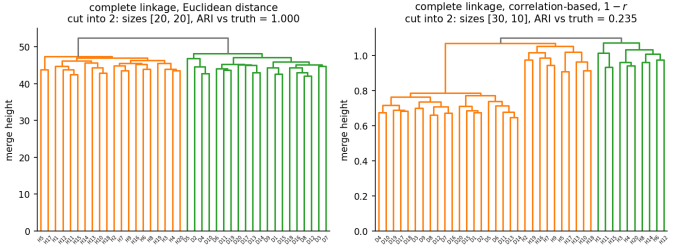
When they disagree

Two shoppers who buy the same categories in the same proportions but one spends ten times more are *far* in Euclidean distance and *identical* in correlation distance. Which is right depends entirely on the question: total spend, or taste?

Takeaway

Standardising the features is itself part of this choice, because scaling changes every Euclidean distance. Also note $1 - r$ needs $p \geq 3$ to mean anything: with $p = 2$ every correlation between two profiles is ± 1.

The measure can matter more than the algorithm



Takeaway

40 tissue samples, 1000 genes, 20 healthy and 20 diseased. With **Euclidean** distance, complete linkage recovers the two groups exactly (sizes 20+20, ARI $= 1.000$) — and so do single and average linkage. With $1 - r$ the same linkage returns $(30, 10)$, ARI $= 0.235$; average gives 0.188 and single gives 0.000. Here the group difference is a shift in 500 genes' levels, which correlation deliberately discards. Swapping the linkage changed little; swapping the *distance* destroyed the signal.

K-means or hierarchical? A decision table

	K-means	Hierarchical
K chosen	before fitting	after, by cutting the tree
Input needed	feature matrix (means must exist)	any dissimilarity matrix
Cost	$O(nKp)$ per iteration, scales to millions	$O(n^2 p)$ memory and time
Randomness	yes — local optima, needs `n_init`	none: deterministic given the linkage
Cluster shape assumed	round, similar size, similar spread	set by the linkage
Nested solutions	no	yes (a restriction as well as a feature)
Main knob to justify	K	linkage *and* dissimilarity

Takeaway

Neither dominates. A common workflow uses hierarchical clustering on a sample to *propose* K and a cluster shape, then K-means on the full data to *assign* everyone — and reports both.

Outline

1 No Response
2 PCA
3 K-means
4 Hierarchical
5 Practice
6 Python Lab
7 Summary

The decisions you must make, and must report

Six choices, none of them made by the data

1. Which **features** enter — adding ten correlated features silently reweights the distance.
2. Whether to **standardise** (usually yes; always state it).
3. The **dissimilarity** — Euclidean, correlation, something domain specific.
4. The **algorithm** and its knobs: K and `n_init`; linkage and cut height.
5. How **outliers** are handled before fitting.
6. How the result is **described** in the write-up.

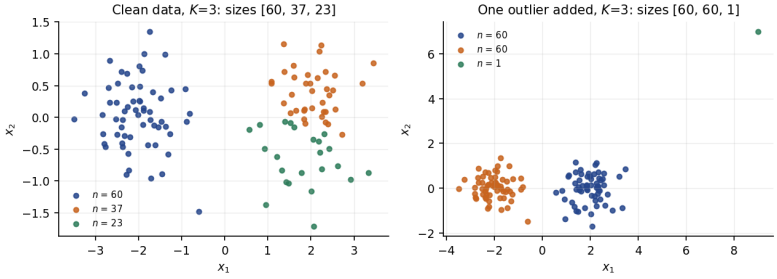
Takeaway

In supervised learning a bad choice shows up as a worse test error. Here it shows up as a different answer that looks exactly as convincing. The report has to carry the choices, because the data cannot.

Live demo — §5 of the notebook

Switch to `advanced_08_unsupervised_lab.ipynb` §5: noise silhouette 0.323, bootstrap ARI 0.68.

Outliers: a single point can buy its own cluster



Common mistake

Left: two real groups of 60; forced to $K = 3$, K-means splits the right-hand group into $37 + 23$. Right: one point added at $(9, 7)$ takes a whole cluster to itself, sizes $(60, 60, 1)$. The squared distance in the objective makes one far point worth hundreds of ordinary ones. Screen for outliers *before* clustering, and treat a near-singleton cluster as a data-quality alarm rather than a discovery.

These methods always return clusters



The error this chapter exists to prevent

Left panel is 200 draws from a *single* two-dimensional Gaussian — there are no clusters. K-means at $K = 3$ returns three tidy, contiguous, roughly equal groups $(67, 59, 74)$, cuts the within-cluster sum of squares from 398.0 to 187.4 (53% "explained"), and scores a silhouette of 0.323 — against 0.494 for the three genuinely separated clusters on the right. There is no threshold on any of these numbers that separates the two panels reliably.

How to report a clustering honestly

The paragraph that should accompany every segmentation

- **Say what was done:** features, scaling, dissimilarity, algorithm, K, `n_init`, seed, achieved objective.
- **Say how stable it is:** re-fit on bootstrap samples or random halves and report the agreement (ARI, or the share of pairs that stay together).
- **Show the profiles** in original units, so a domain expert can judge whether the groups are recognisable.
- **Show one alternative** — a different K or linkage — and **call the result a hypothesis**, naming the experiment or downstream metric that would confirm it.

Measuring stability on this chapter's example

Re-fit $K = 3$ K-means on 200 bootstrap resamples of the 50 states and compare each fit's assignment of all 50 states with the full-data fit: mean adjusted Rand index 0.68 (median 0.77). Reproducible enough to report — and far from the 1.0 that "three types of state" would imply.

The wording test

"We identified three customer types" claims a finding you cannot support. "A three-cluster solution on standardised spend and tenure features gives three interpretable profiles, stable at ARI 0.68 across bootstrap re-fits; a matched-control campaign will test whether the segments respond differently" claims exactly what you have.

The bridge to module A7 (multiple testing)

Turning structure into something testable

The gene data makes the link explicit. Clustering *suggested* that the 40 samples fall into two groups. To claim a finding you must name the genes and test them — 1000 two-sample t-tests, one per gene. Then you are squarely in module A7: 159 genes have $p < 0.05$ (of which ~ 50 are expected by chance alone), while 105 survive Bonferroni at $0.05/1000$, and 91 of the 100 smallest p-values fall in genes 501–1000 — exactly the block that was constructed to differ.

Common mistake

Testing the genes that *defined* the clusters and reporting the p-values as if the groups had been given in advance. The clustering already used those genes; this is post-selection inference, and the p-values are optimistic even after Bonferroni. An honest version needs an independent sample — or the selective-inference machinery of module A7's appendix.

Extended Exercise 12.2 — Clustering pure noise [Python]

Extended exercise (15 min)

Show that the standard diagnostics cannot detect the absence of clusters.

1. Simulate $n = 200$ points from a single standard bivariate normal with `rng = np.random.default_rng(2024)`, and a second data set of three genuinely separated clusters (means $(-2.5, 0)$, $(2.5, 0)$, $(0, 2.5)$, unit variance).
2. For $K = 1, \ldots, 8$ record $W(K)$ and, for $K \geq 2$, the mean silhouette width on both data sets.
3. Report $W(3)/W(1)$ and the silhouette at $K = 3$ for each, and state at which K the silhouette peaks on the noise.
4. Propose a diagnostic that *would* have flagged the noise, and say what it costs.

Run this live

The worked solution sits in `advanced_08_unsupervised_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Extended Exercise 12.2 — Clustering pure noise [Python]*.

Solution — Extended Exercise 12.2 (1/2)

Solution — code

```
from sklearn.cluster import KMeans # same tool on both data sets
from sklearn.metrics import silhouette_score # separation, per point

rng = np.random.default_rng(2024) # same seed as the slide figure
noise = rng.normal(size=(200, 2)) # ONE Gaussian: no clusters at all
mu = np.array([[-2.5, 0.], [2.5, 0.], [0., 2.5]]) # 3 true centres
lab = rng.integers(0, 3, 200) # true group of each point
real = mu[lab] + rng.normal(size=(200, 2)) # three separated clusters
km = lambda X, K: KMeans(n_clusters=K, n_init=50, random_state=0).fit(X)

for name, X in [('noise', noise), ('real ', real)]: # identical recipe
    W = {K: km(X, K).inertia_ for K in range(1, 9)} # W falls either way
    sil = {K: silhouette_score(X, km(X, K).labels_) for K in range(2, 9)}
    peak = max(sil, key=sil.get) # the K a scree/sil rule would pick
    print(f'{name} W(3)/W(1)={W[3]/W[1]:.3f} sil(3)={sil[3]:.3f} '
          f'peak K={peak} ({sil[peak]:.3f})') # noise answers too
```

Solution — Extended Exercise 12.2 (2/2)

Solution — output and what it means

Printout.

```
noise W(3)/W(1)=0.471 sil(3)=0.323 peak K=8 (0.337)
real  W(3)/W(1)=0.248 sil(3)=0.494 peak K=3 (0.494)
```

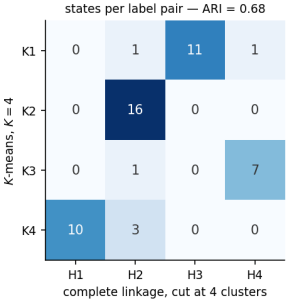
- **Parts 2–3.** Three clusters of nothing still cut the objective by 52.9% (398.0 $\rightarrow$ 187.4) and score a silhouette of 0.323, a value many practitioners would call "reasonable structure". On the noise the silhouette *rises* again to 0.337 at $K = 8$: it never says "stop". Only the comparison with the real data (0.494, peaking exactly at $K = 3$) exposes it — and in an application there is no real data to compare with.
- **Part 4 — a diagnostic that works.** Compare the observed $\log W(K)$ with its distribution under a simulated *no-cluster* reference: the gap statistic. Cost: you must specify the null, it is far more computation, and it has power only for well-separated clusters. There is no free validation here.

Common mistake: quoting "$K = 3$ explains 53% of the variation" as evidence of three groups. On this noise it is literally true and completely empty.

Outline

1 No Response
2 PCA
3 K-means
4 Hierarchical
5 Practice
6 Python Lab
7 Summary

Do the two methods find the same structure?



Takeaway

K-means ($K = 4$) against complete linkage (cut at 4) on scaled USArrests: most states land in matching pairs and the adjusted Rand index is 0.68 — the two algorithms broadly agree on the structure without agreeing on every state. Crosstabbing two clusterings is the same validation move the lab makes on NCI60.

Python lab — run it live in the notebook

Companion notebook: `advanced_08_unsupervised_lab.ipynb`

The code in this section is meant to be **demonstrated live**. Open the companion Jupyter notebook and run it cell by cell:

- §1, *PCA on USArrests* — loadings, scores, PVE, the biplot and the scree plot, and the scaled-vs-unscaled comparison;
- §2, *K-means* — $K = 1, \ldots, 4$, cluster profiles in original units, and the local-optimum experiment;
- §3, *Hierarchical clustering* — five linkages, cutting the tree, and how much the partition moves;
- §4, *High-dimensional data: gene expression* — PCA and clustering on the bundled 1000×40 matrix (with the optional `NCI60` branch);
- §5, *Practical issues: outliers, and clusters in pure noise* — plus the bootstrap stability check.

Data loads via the `ISLP` package or the bundled CSV files; §6, *Exercises* ends the notebook with hands-on tasks, after the worked solutions to every `[Python]` exercise in this deck.

PCA in scikit-learn

```
import numpy as np, pandas as pd # arrays and frames
from sklearn.decomposition import PCA # the PCA estimator
from sklearn.preprocessing import StandardScaler # mean 0, sd 1 per column

USA = load('USArrests', index_col=0) # 50 states x 4 features
Xs = StandardScaler().fit_transform(USA) # standardise: mean 0, sd 1 per column
pca = PCA().fit(Xs) # all min(n-1, p) directions

print(np.round(pca.explained_variance_ratio_, 4)) # [0.6201 0.2474 0.0891 0.0434]
print(np.round(np.cumsum(pca.explained_variance_ratio_), 4)) # 2 PCs = 86.8%
loadings = pd.DataFrame(pca.components_.T, index=USA.columns,
                        columns=[f'PC{i+1}' for i in range(4)]) # rows = variables
print(loadings.round(3)) # signs may be flipped: fine
Z = pca.transform(Xs) # scores: states in PC space
```

How to read this

`components_` holds the loadings, one *row* per PC (hence `.T`); `explained_variance_ratio_` is the PVE. Without the scaler PC1 would be `Assault` alone at 96.6%.

K-means, and proving the local-optimum problem

```
from sklearn.cluster import KMeans # Lloyd's algorithm

km = KMeans(n_clusters=3, n_init=50, random_state=0).fit(Xs) # 50 restarts
print(round(km.inertia_, 3)) # W = 79.922, the objective
print(USA.assign(cluster=km.labels_) # profiles in RAW units
        .groupby('cluster').mean().round(1)) # cluster means per variable

w = [KMeans(n_clusters=4, n_init=1, random_state=s).fit(Xs).inertia_
     for s in range(20)] # twenty single-start runs
print(np.round(sorted(w), 3)) # 57.554 best, 77.856 worst
print('distinct local optima:', len(set(np.round(w, 3)))) # 9 different answers
```

What `n_init` is buying

Lloyd's algorithm only ever reaches a *local* minimum of W, and which one is fixed by the random starting centroids. With `n_init=1` the twenty seeds land on **nine** different values, 57.554 up to 77.856 — 35% apart on one data set. `n_init=50` keeps the smallest W of 50 starts, and `random_state` makes that choice reproducible; report both.

Hierarchical clustering with scipy

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import squareform # square D -> condensed

for method in ['complete', 'average', 'single', 'centroid']:
    Zl = linkage(Xs, method=method) # n-1 merges: i, j, height, size
    lab = fcluster(Zl, 3, criterion='maxclust') # slice the tree, 3 branches left
    print(f'{method:9s} sizes {np.bincount(lab)[1:]} top fusion {Zl[-1,2]:.2f}')

Dcorr = 1 - np.corrcoef(Xs) # 1-r: shape, not level
Zc = linkage(squareform(Dcorr, checks=False), 'complete') # needs condensed form
dendrogram(Zl, labels=USA.index.tolist(), leaf_font_size=6) # HEIGHT is the message
```

What `linkage` returns, and what the cut does

`Zl` is the $(n-1) \times 4$ *linkage matrix*: one row per fusion — the two things joined, the **height** at which they joined, the size of the new group. On raw data it assumes Euclidean distance, so any other dissimilarity must arrive condensed (`Zc`). `fcluster` re-fits nothing; it slices that same tree low enough to leave 3 branches: $(31, 11, 8)$, $(30, 19, 1)$, $(48, 1, 1)$, $(30, 19, 1)$.

Outline

1 No Response

2 PCA

3 K-means

4 Hierarchical

5 Practice

6 Python Lab

7 Summary

Module A8 in one slide

What we accomplished

Finding structure in X alone — and reporting it honestly, without a test error to hide behind.

- Why unsupervised learning has no validation, and what can be borrowed instead: stability, interpretability, a downstream supervised task.
- PCA: loadings, scores, orthogonality, PVE, the scree plot, the biplot, sign indeterminacy, and the fact that scaling changes everything.
- K-means: the within-cluster objective, Lloyd's algorithm, local optima measured rather than asserted, and `n_init`.
- Hierarchical clustering: the dendrogram, height versus horizontal position, four linkages, and the dissimilarity measure.
- Practice: outliers, clusters in pure noise, and the wording of an honest report.

Key formulas at a glance (1/2)

Standardising $\tilde{x}_{ij} = (x_{ij} - \bar{x}_j)/s_j$; PCA on standardised data = PCA on the correlation matrix.

First component $\max_{\phi_1} \frac{1}{n}\sum_i \left(\sum_j \phi_{j1} x_{ij}\right)^2$ s.t. $\sum_j \phi_{j1}^2 = 1$; solution = top eigenvector of the covariance (or correlation) matrix.

Scores $z_{im} = \sum_{j=1}^{p} \phi_{jm} x_{ij}$, with x centred.

Orthogonality $\phi_m^\top \phi_{m'} = 0$ and $\mathrm{Cov}(z_{\cdot m}, z_{\cdot m'}) = 0$ for $m \neq m'$.

Variance partition $\lambda_m = \mathrm{Var}(z_{\cdot m})$ and $\sum_{m=1}^{p} \lambda_m = \sum_{j=1}^{p} \mathrm{Var}(X_j)$ $(= p$ if standardised$)$.

PVE $\mathrm{PVE}_m = \lambda_m / \sum_{m'} \lambda_{m'} = \sum_i z_{im}^2 / \sum_j \sum_i x_{ij}^2$; cumulative $\sum_{m \le M} \mathrm{PVE}_m$.

Sign indeterminacy $\phi_m \mapsto -\phi_m$, $z_{im} \mapsto -z_{im}$, λ_m unchanged.

Low-rank view $\min_{a,b} \sum_{ij} \left(x_{ij} - \sum_{m \le M} a_{im} b_{jm}\right)^2$ is solved by $a_{im} = z_{im}$, $b_{jm} = \phi_{jm}$; residual SS $= n \sum_{m > M} \lambda_m$.

Two-variable case $\mathrm{R} = \left(\begin{smallmatrix} 1 & r \\ r & 1 \end{smallmatrix}\right) \Rightarrow \lambda_{1,2} = 1 \pm r$, $\phi_{1,2} = \frac{1}{\sqrt{2}}(1, \pm 1)^\top$, $\mathrm{PVE}_1 = (1 + r)/2$.

Key formulas at a glance (2/2)

K-means objective $\min_{C_1..C_K} \sum_k W(C_k)$ with $W(C_k) = \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_j (x_{ij} - x_{i'j})^2$.

Centroid form $W(C_k) = 2 \sum_{i \in C_k} \sum_j (x_{ij} - \bar{x}_{kj})^2$, $\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$ (software reports the version without the 2).

Assignment step $i \mapsto \arg\min_k \|x_i - \bar{x}_k\|^2$; the mean minimises $\sum_{i \in C_k} \|x_i - c\|^2$, which is why both steps decrease W.

SS decomposition $\mathrm{TSS} = \sum_i \sum_j (x_{ij} - \bar{x}_j)^2 = W + \mathrm{BSS}$; "variance explained" $= 1 - W/\mathrm{TSS}$ (rises with K by construction).

Euclidean dissimilarity $d(i, i') = \sum_j (x_{ij} - x_{i'j})^2$.

Correlation dissimilarity $d(i, i') = 1 - r_{ii'}$, $r_{ii'}$ the correlation of the two feature profiles ($p \geq 3$).

Linkage complete $\max_{i \in A, i' \in B} d(i, i')$; single min; average $\frac{1}{|A||B|} \sum \sum d(i, i')$; centroid $\|\bar{x}_A - \bar{x}_B\|^2$.

Silhouette $s_i = \dfrac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$, with a_i the mean distance to i's own cluster and b_i the smallest mean distance to another cluster.

Agreement of two partitions adjusted Rand index: 0 for chance agreement, 1 for identical partitions.

Vocabulary checklist

Term	Meaning
Loading ϕ_{jm}	weight of feature j in component m; defines a direction
Score z_{im}	coordinate of observation i along component m
PVE	share of total variance carried by a component
Scree plot	PVE against component number; the "elbow" is a heuristic
Biplot	scores and loading arrows in one picture
Sign indeterminacy	ϕ_m and $-\phi_m$ are the same component
Within-cluster variation W	the K-means objective; `inertia_`
Local optimum	a partition no single move improves; not necessarily the best
`n_init`	number of random starts kept for the best objective
Dendrogram	tree of fusions; *height* carries the information
Linkage	rule for the dissimilarity between two clusters
Chaining	single linkage's habit of growing one long cluster
Inversion	a fusion drawn below one of its children (centroid linkage)
Silhouette width	how much better an observation fits its own cluster
Gap statistic	compares $W(K)$ with a no-cluster reference distribution

Decision rules of thumb

- Features in different units $\Rightarrow$ **standardise**, and say so. `USArrests` unscaled gives PC1 = Assault at 96.6%.
- Want a picture of p variables $\Rightarrow$ PCA, report cumulative PVE, and admit what the projection discards.
- Choosing M or K $\Rightarrow$ heuristics propose, the domain decides. Never call an elbow a result.
- Running K-means $\Rightarrow$ `n_init` ≥ 20, fixed seed, report the objective.
- n small enough for an $n \times n$ matrix and no idea what K is $\Rightarrow$ hierarchical; then justify linkage *and* dissimilarity.
- Profiles matter more than levels $\Rightarrow$ correlation-based dissimilarity; levels matter $\Rightarrow$ Euclidean.
- Before believing any partition $\Rightarrow$ re-fit on halves or bootstraps and report the agreement.
- Writing it up $\Rightarrow$ "hypothesis", plus the experiment that would test it.

Common pitfalls

Don't

1. Run PCA on unstandardised variables in different units and celebrate a huge PVE.
2. Assume your figure is wrong because a component's sign differs from the book's.
3. Read horizontal proximity in a dendrogram as similarity.
4. Run K-means once and report the partition as *the* partition.
5. Compare W across different K and call the decrease "validation".
6. Cluster with outliers still in, then report the singleton cluster as a segment.
7. Present clusters found in a single unimodal cloud as customer types.
8. Test the very features that defined the clusters and quote the p-values as if the groups were given in advance.

Self-check questions

Quick self-assessment

1. Why can a held-out sample not validate a clustering?
2. What are the two constraints defining the second principal component?
3. `USArrests` unscaled gives $\mathrm{PVE}_1 = 96.6\%$; scaled it gives 62.0%. Which do you report, and why?
4. Your PC2 loadings are exactly the negatives of a colleague's. Who is wrong?
5. Two runs of K-means at $K = 4$ give $W = 57.554$ and $W = 58.242$. What do you do?
6. Complete linkage cut at 3 gives $(31, 11, 8)$; single linkage gives $(48, 1, 1)$. Which is correct?
7. Pure noise gives a silhouette of 0.323 at $K = 3$. What follows?

Answers

(1) Assigning a held-out point to its nearest centroid *defines* its label, so the "error" is zero by construction. (2) Unit norm and orthogonality to ϕ_1. (3) The scaled one — otherwise you are reporting units. (4) Neither: signs are arbitrary. (5) Keep $W = 57.554$ and raise `n_init`; the other run is simply worse. (6) Both, for their own assumption about cluster shape — the question is which assumption you can defend. (7) That the silhouette cannot detect the absence of clusters, so it must not be used as evidence that clusters exist.

Five things to remember

Module A8 in five lines

1. **No response means no test error.** Every answer here is a hypothesis; nothing in the data can call it wrong.
2. **Scaling is a decision, not a formality.** On `USArrests` it moves PVE_1 from 62.0% to 96.6% and turns PC1 into one variable.
3. **K-means finds a local optimum.** Twenty single starts at $K = 4$ gave nine different answers, the worst 35% off; `n_init` is not optional.
4. **In a dendrogram, only height means anything** — and the linkage and the distance can matter more than the algorithm.
5. **These methods always return clusters**, including from pure noise. Report the choices, the stability, and the test you would run next.

What's next

- **Module A7 — Multiple Testing**: the gene example ended with 1000 t-tests and ~ 50 false discoveries expected by chance — A7 is the machinery (FWER, Holm, BH) that turns suggested structure into defensible claims.
- **Back to Chapter 6**: principal component regression is this module's PCA put to supervised work — the scores as predictors.

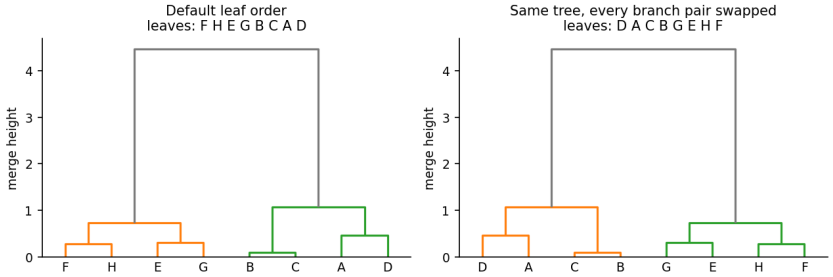
Appendix — what is in here, and why

Nothing in the appendix is needed to follow the chapter: each slide is a formal derivation, a longer worked exercise, or a side topic. Read it when you want the full story, or when a later chapter sends you back here.

Contents of the appendix

- **The same tree, drawn two ways** — a dendrogram has 2^{n-1} equally valid drawings; the rule is stated in the main deck, and the two-renderings figure lives here.
- **The two forms of the K-means objective** — the algebraic identity that turns the all-pairs definition into the centroid form. Quoted as a fact in the main deck because the algebra is a distraction there.
- **PCA as the best low-rank approximation** — why maximising variance and minimising reconstruction error are the same problem, and the residual formula.
- **PCA with missing values** — the matrix-completion algorithm, needed in practice and beyond what the main deck needs.
- **The silhouette width** — the formula, and what it looks like on `USArrests`.
- **Extended Exercise 12.3** — a full clustering workflow on the bundled gene-expression data. It needs 20 minutes at the keyboard, so leave it until you have the notebook open.

The same tree, drawn two ways



The commonest dendrogram error

Both panels are the *identical* clustering: every merge height is the same, only the left/right order of each branch pair was swapped. Leaf order changes from `F H E G B C A D` to `D A C B G E H F`. In the left panel G looks "next to" B; in the right it does not. If your conclusion changes between these two pictures, your conclusion was about the drawing.

Derivation: the two forms of the K-means objective

Claim

$$\frac{1}{|C_k|}\sum_{i,i'\in C_k}\sum_{j=1}^{p}(x_{ij}-x_{i'j})^2 = 2\sum_{i\in C_k}\sum_{j=1}^{p}(x_{ij}-\bar{x}_{kj})^2.$$

Fix a feature j, write $n_k = |C_k|$ and $\bar{x} = \bar{x}_{kj}$. Expand around the mean:

$$\sum_{i,i'\in C_k}(x_{ij}-x_{i'j})^2 = \sum_{i,i'}\big[(x_{ij}-\bar{x})-(x_{i'j}-\bar{x})\big]^2 = \sum_{i,i'}\Big[(x_{ij}-\bar{x})^2+(x_{i'j}-\bar{x})^2-2(x_{ij}-\bar{x})(x_{i'j}-\bar{x})\Big].$$

The first two double sums each give $n_k\sum_i(x_{ij}-\bar{x})^2$; the cross term factorises into $-2\big(\sum_i(x_{ij}-\bar{x})\big)^2$, which is **zero** because deviations from the mean sum to zero. Hence $\sum_{i,i'\in C_k}(x_{ij}-x_{i'j})^2 = 2n_k\sum_{i\in C_k}(x_{ij}-\bar{x}_{kj})^2$, and dividing by n_k and summing over j gives the claim.

Takeaway

Two consequences: the objective depends on the data only through squared Euclidean distances, and the centroid form shows why re-computing means can never increase W — the mean is the minimiser of $\sum_i\|x_i-c\|^2$.

PCA as the best low-rank approximation

Eckart–Young, in the notation of this chapter

For any $M \le \min(n-1, p)$,

$$\min_{A \in \mathbb{R}^{n \times M},\, B \in \mathbb{R}^{p \times M}} \sum_{i=1}^{n} \sum_{j=1}^{p} \Big(x_{ij} - \sum_{m=1}^{M} a_{im} b_{jm} \Big)^2 \;=\; n \sum_{m=M+1}^{p} \lambda_m,$$

attained at $a_{im} = z_{im}$, $b_{jm} = \phi_{jm}$ (unique up to rotation within the retained subspace, and up to the sign of each component).

How to read this

- Total sum of squares is $n \sum_m \lambda_m$; the first M components account for $n \sum_{m \le M} \lambda_m$, so the leftover is the tail of the eigenvalues.
- Dividing through, the *fraction* of squared error left after M components is exactly $1 - \sum_{m \le M} \mathrm{PVE}_m$. For `USArrests` at $M = 2$ that is 13.2%.
- "Rotation within the subspace" is why factor analysis can rotate loadings (varimax) without changing the fit: the *subspace* is identified, the axes inside it only by the variance-ordering convention.

PCA with missing values: matrix completion

Algorithm (hard-impute)

1. Fill each missing x_{ij} with the column mean $\bar{x}_j$ of the observed entries.
2. Compute the rank-M PCA approximation $\hat{x}_{ij} = \sum_{m \le M} z_{im}\phi_{jm}$ of the completed matrix.
3. Replace *only the missing* entries by their $\hat{x}_{ij}$; keep observed entries as they are.
4. Repeat 2–3 until the fitted values stop changing.

How to read this

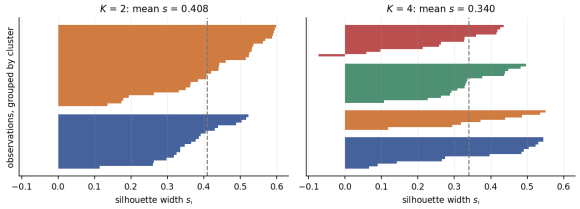
Each iteration solves the low-rank problem of the previous slide over the observed entries only, so the objective decreases monotonically — the same "guaranteed to converge, not guaranteed to be optimal" situation as K-means. M must be chosen, and the method assumes entries are missing for reasons unrelated to their values. Under another name this is the classical recommender engine: a user-by-item rating matrix is mostly missing, and low-rank completion of it puts M latent factors in place of taste.

The silhouette width

Definition

For observation i in cluster C_k, let a_i be its mean distance to the other members of C_k and $b_i = \min_{k' \neq k}$ (mean distance to members of $C_{k'}$). Then

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1].$$



Takeaway

On standardised `USArrests` the mean silhouette is 0.408 at $K = 2$ and 0.340 at $K = 4$, so the criterion prefers $K = 2$. Negative s_i marks observations that sit closer to another cluster than their own. Useful for *ranking* candidate K; useless for deciding whether any clustering is real (see the noise example).

Extended Exercise 12.3 — A full workflow on gene expression [Python]

Extended exercise (15 min)

`Ch12Ex13.csv` holds 1000 genes (rows) measured on 40 tissue samples (columns); the first 20 samples are healthy and the last 20 diseased — a label you may use *only* to score your answer at the end, never to fit.

1. Cluster the 40 *samples* with complete, average and single linkage under (i) Euclidean distance and (ii) correlation-based distance $1 - r$. Cut each tree into two clusters.
2. Score all six partitions against the true labels with the adjusted Rand index. Which decision — linkage or dissimilarity — mattered more?
3. Run PCA on the samples. What does PC1 do, and how much variance does it explain?
4. Which genes differ between the groups? Rank them and count how many survive a Bonferroni threshold at $0.05/1000$.

Run this live

The worked solution sits in `advanced_08_unsupervised_lab.ipynb` under *Lecture exercises — worked Python solutions*: the cell headed *Extended Exercise 12.3 — A full workflow on gene expression [Python]*.

Solution — Extended Exercise 12.3 (1/3)

Solution — code: clustering under two dissimilarities

```
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster # build, then cut
from scipy.spatial.distance import squareform # linkage wants D condensed
from sklearn.metrics import adjusted_rand_score # 1 = exact, 0 = chance

G = load('Ch12Ex13', header=None).values # 1000 genes x 40 samples
X = G.T # cluster the SAMPLES: 40 x 1000
truth = np.array([0]*20 + [1]*20) # for SCORING only, never for fitting

for nm, arg in [('eucl', X), ('1-r', squareform(1-np.corrcoef(X), checks=False))]:
    for m in ('complete', 'average', 'single'): # 3 linkages x 2 distances
        lab = fcluster(linkage(arg, method=m), 2, criterion='maxclust') # cut in 2
        print(nm, m, np.bincount(lab)[1:], round(adjusted_rand_score(truth, lab), 3))
```

Solution — Extended Exercise 12.3 (2/3)

Solution — code: PCA and the per-gene tests, and the printout

```
from scipy import stats # one t-test per gene
from sklearn.decomposition import PCA # centred, not standardised

p = PCA().fit(X - X.mean(0)); Z = p.transform(X - X.mean(0)) # PCA on the samples
print('PVE1', round(p.explained_variance_ratio_[0], 3), # 0.127: small, not useless
      ' PC1 group means', round(Z[:20, 0].mean(), 2), round(Z[20:, 0].mean(), 2))
t, pv = stats.ttest_ind(G[:, 20:], G[:, :20], axis=1) # 1000 two-sample t-tests
print((pv < 0.05).sum(), (pv < 0.05/1000).sum()) # nominal, then Bonferroni
```

```
eucl complete [20 20] 1.0    1-r complete [30 10] 0.235
eucl average  [20 20] 1.0    1-r average  [31  9] 0.188
 eucl single   [20 20] 1.0     1-r single   [39  1] 0.0
  PVE1 0.127  PC1 group means -11.75 11.75    159 105
```

How to read this

PCA runs on *centred*, not standardised, data — the genes already share one scale. `G` is genes × samples, so `axis=1` tests each gene across the 40 columns.

Solution — Extended Exercise 12.3 (3/3)

Solution — interpretation

- **Parts 1–2.** Euclidean distance recovers the truth under all three linkages; $1 - r$ under none. The **dissimilarity** mattered enormously, the linkage barely at all: the diseased group differs by a *level shift* in genes 501–1000 (mean absolute difference 0.61 there versus 0.30 elsewhere), and $1 - r$ is engineered to ignore levels.
- **Part 3.** PC1 explains only 12.7% yet separates the groups completely (means -11.75 vs $+11.75$): with $p \gg n$ a modest PVE can be the whole story.
- **Part 4.** 159 genes have $p < 0.05$, but ~ 50 are expected under the global null. 105 survive Bonferroni at 5×10^{-5}, and 91 of the 100 smallest p-values lie in genes 501–1000 — module A7, and the only part of this analysis with a guarantee attached.

Common mistake

Using `truth` anywhere before step 2. If the labels are available you have a *supervised* problem; clustering is what you do when they are not.